Cours 02 : Notions de base en Python

.

I. Les commentaires

En développement, il est nécessaire de commenter régulièrement son code. Les commentaires permettent d'**expliquer le code**, d'en comprendre son fonctionnement. Ils ne sont pas interprétés et donc ne s'affiche pas lors de l'éxecution du code. Ils ne servent qu'entre les développeurs.

En Python, le symbole **#** est un commentaire. Le commentaire commence par le symbole # puis s'étend jusqu'à la fin de la ligne.

Exemple :

```
# commentaire
```

En Python, il est possible d'écrire des commentaires en utilisant **''' (triple guillemets)** de part et d'autre du commentaire.
Il s'agit d'un commentaire multiplignes souvent utilisé dans les documentations.

Exemple :

```
'''
commentaire
multiplignes
'''
```

.

II. Les variables

.

1. Définition

Une variable est une **données ponctuelle stockée dans la mémoire de l'ordinateur**.
Elle a deux caractéristiques : un **nom** et une **valeur**.

.

2. Affection en Python

Pour créer une variable dans un langage de programmation, il est nécessaire de faire :

- la **déclaration** de la variable : on lui donne un **nom** et son **type**.
- l'**initialisation** de la variable : on lui donne une **valeur**.

En Python, ces deux étapes se font en même temps. On parle d'affectation de la variable. L'**opérateur d'affectation** en Python est **=**

> **Syntaxe** affection d'une variable : ***nom = valeur***

Exemples :

```
# Affectation des variables
var_1 = 56
var_2 = "Hello"
```

Python est un langage au **typage dynamique** : Il assigne le type de la variable en fonction du type de la première valeur lors de l'affectation.

.

3. Nomenclature

Pour nommer vos variables, il est nécessaire de respecter les règles suivantes:

- **Eviter d'utiliser des mots clés** de Python (if, for, print, etc.)
- **Ne pas commencer par un chiffre**.
- Le **nom** de la variable doit être **explicites** pour rendre le programme plus lisible.
- Python est **sensible à la casse** (majuscules/minuscules) donc la variable *age* est différente de la variable *Age*
- **Ne pas utiliser d'espaces**. Si le nom de votre variable doit contenir deux mots vous pouvez utiliser le style ***snake_case*** : les mots sont en **minuscule** et séparé par un **underscore** (tiret du bas).

Exemples :

```
age_utilisateur = 20
reponse = True
```

4. Propriétés

- **Ecrasement** : Une variable ne peut contenir qu'une seule valeur. Si on affecte une nouvelle valeur à une varialbe alors l'ancienne sera effacée et remplacée par la nouvelle.

  Exemple :

  ```
  age = 4
  age = 3
  # La variable age possède la valeur 3.
  ```

- **Copie** : Il est possible de copier la valeur d'une variable dans une autre variable.

  Exemple :

  ```
  taille = 168
  mesure = taille
# La variable mesure possède la valeur 168.
  ```

- **Affectation en parallèle** :
  Il est possible de d'affecter plusieurs variables en une seule ligne.

  Exemple :

  ```
  age, taille = 34, 178
  # Les variables age et taille possèdent les valeurs respectives de 34 et
178.
  ```

- **Incrémentation** :
  L'opérateur **+= effectue une addition puis affecte le résultat à la même variable.**
  Cela fonctionne aussi avec les **autres opérateurs arithmétiques** (*voir le tableau des opérateurs arithmétiques plus loin dans ce cours.*)

  Exemples :

  ```
  age = 15
  age += 2
  # La variable age possède la valeur 17.

  taille = 164
  new_taille /= 2
  # Les variables taille et new_taille possèdent les valeurs respectives
de 164 et 82.
  ```

III. Les fonctions print() et input()

.

1. La fonction print()

La fonction print() permet d'**AFFICHER** l'argument qui est entre les parenthèse et ajoute un retour à la ligne.

Cette fonction permet d'afficher :

- la **valeur** d'une variable quel que soit son type.
- du **texte** en utilisant des guillemets.
- du **texte et** la **valeur** d'une variable séparé par une virgule.

Exemple :

```
distance = 35
print(distance)
print("La distance")
print("La distance est de ", distance)
    # Le programme affiche :
    # 35
    # La distance
    # La distance est de 35
```

Le **retour à la ligne** est **par défaut** mais il est possible de le changer en utilisant l'argument ***end*=**.

Exemple :

```
distance = 35
print(distance, end=":")
print("La distance")
    # Le programme affiche : 35:La distance
```

Pour aller plus loin : Renseignez-vous sur l'utilisation des **f-strings** en Python.

.

2. La fonction input()

Il est parfois nécessaire de **DEMANDER** à l'utilisateur de saisir des données avant de commencer l'algorithme. En Python pour demander des données à l'utilisateur, on utilise la fonction **input()**.

Exemple :

```
code = input("Entrez votre code : ")
```

Le programme affiche ***Entrer votre code*** puis attend que l'utilisateur saisisse une donnée pour continuer le programme. Cette donnée est stockée dans la variable ***code***.

Remarque : Par défaut, **la fonction input() renvoie toujours une chaîne de caractère**.

.

IV. Les types de données

En Python, pour connaître le type d'une variable vous devez utiliser la fonction ***type()***.

.

1. Les numériques

- **integer (int)** : les nombres entiers.

Exemple :

```
int_1 = 3
int_2 = -65
```

- **float** : les nombres décimaux.
  Attention la partie entière et la partie décimale doivent être séparée par un **point** et pas une virgule.

Exemple :

```
float_1 = 58.47
float_2 = -4.9
```

Les opérateurs arithmétiques :

Opérations	Symboles	Exemples
addition	+	3 + 2 donne 5
soustraction	-	5 - 2 donne 3
multiplication	*	4 * 3 donne 12
division	/	27 / 3 donne 9
puissance	**	6 ** 2 donne 36
quotient de la division	//	45 // 2 donne 22
reste de la division	%	45 % 2 donne 1

.

2. Les chaînes de caractères

- **string (str)** : le texte
  Les chaînes de caractères sont entourées par des **guillemets** (simple ou double quote).

Exemple :

```
str_1 = "Hello"
str_2 = 'Bonjour'
```

Les opérations sur les chaînes de caractères :

- Il est possible de connaître le nombre de caractère dans une chaîne de caractères en utilisant la fonction ***len()***.

Exemple :

```
str_1 = "Hello"
print(len(str_1))
    # Le programme affiche : 5
```

- La **concaténation** : l'opérateur **+** entre **deux chaînes de caractères** permet des les **concaténer**. Cela permet de les coller ensemble.

Exemple :

```
print("Bonne"+"journée!")
    # Le programme affiche : Bonnejournée!
```

- La **duplication** : l'opérateur * entre un **integer** et un **string** permet de dupliquer le string autant de fois que l'indique l'integer.

Exemple :

```
print("Bonjour"*3)
    # Le programme affiche : BonjourBonjourBonjour
```

Remarque : Il est impossible d'utiliser l'opérateur **+** entre un string et un integer. Cela provoque une erreur.

.

3. Les collections

Une **collection de données** permet de stockées plusieurs valeurs dans une seule structure. Ces valeurs sont toujours séparées par une **virgule**.
Les collections permettent de mieux orgsaniser les données.
Pour connaître le **nombre d'éléments** dans une collection, vous devez utiliser la fonction ***len()***.
En Python il en existe trois types de collections de données :

- **list** : les listes
  Pour créer une liste on utilise les **crochets**.

Exemple :

```
list_1 = [2,9,45]
list_2 = [78, "trois", 5.6]
```

- **tuple** : les tuples
  Pour créer un tuple on utilise les **parenthèses**.
  Les tuples sont très similaires aux listes mais les valeurs dans le tuples ne peuvent pas être modifiées contrairement aux listes.

Exemple :

```
tuple_1 = (48,"Bonjour")
```

- **dict** : les dictionnaires.
  Pour créer un dictionnaire on utilise les **accolades**.
  Les dictionnaires permettent de stockées un couple de données (la clé et la valeur).
  Exemple :

```
dict_1 = {"France" : "Paris", "Espagne" : "Madrid"}
```

Remarque : Les collections seront étudiées dans les chapitres suivantes.

.

4. Les booléens

- **booleen (bool)** : les booléens
  Il existe deux booléens : **True** et **False**.
  Ces booléens sont utilisés dans les **structures conditionnelles (if)**.
  Il est possible de faire des **tests de comparaison** ayant pour réponse True ou False : ce sont des booléens.
  .

Les opérateurs de comparaison :

Opérateurs	Explication
x == y	x est **égal** à y
x != y	x est **différent** de y
x > y	x est **supérieur** à y
x < y	x est **inférieur** à y
x >= y	x est **supérieur ou égal** à y
x <= y	x est **inférieur ou égal** à y

Remarque : Les booléens seront revus dans le chapitre sur les structures conditionnelles.

.

5. Conversions de type

Il est possible de convertir (changer) le type d'une variable en utilisant les fonctions : ***int(), float(), str(), list(), tuple(), dict(), bool()***

Exemple :

```
var_1 = 36.3
var_2 = int(var_1)
print(var_2)
    # Le programme affiche : 36
```